

Brick-by-Brick Use Case: Address Resolution

Use Case: Address Resolution

Given multiple overlapping, differently-shaped feeds/datasets of company addresses, produce one dataset where each real-world entity appears exactly **once**, with its **best** address — and be able to explain *why* you chose it.

3. Industry & business context

Industry: Supply-chain intelligence / global trade.

Altana Technologies uses AI to map global supply chains — who makes what, where it comes from, how it moves, and how it works economically — by fusing shipment records, corporate registries, cargo manifests, and open-source intelligence. The primary way of identifying any logistical location is by **address**.

An address is a human-readable set of spatial instructions meant to pinpoint a physical location on Earth. It feels trivial — type your street, a box appears on your doorstep — but to a computer it is a chaotic, unstandardized artifact shaped by local culture, shifting geography, language, and human error. Software thrives on rigid structures, unique primary keys, and deterministic logic; addresses are messy historical artifacts that refuse to provide them.

The core issue is that there is **no single global standard for defining location**. In the US and Western Europe we rely on hierarchical line items: building number, street name, city, region, postal code. Cross a border and those assumptions collapse — Japan organizes by administrative district and block number rather than street name; parts of Latin America can use casual directions like "200 meters south of the old fig tree, blue house on the left." Add free-text fields, multilingual translations, character-set variation, and typos, and the simple question "*where is this facility?*" becomes a major data-engineering challenge. This problem — **entity resolution** — sits at the very foundation of the business: everything downstream depends on knowing whether two records point to the same place, and whether a location is even real.

4. The problem / business question

Do two records refer to the same place, and is a given address even legitimate?

Multiple data providers deliver feeds of companies and their addresses. The feeds overlap heavily, use different schemas, disagree with each other, and are full of noise. The task is to fuse them into a single canonical set of entities, each with a defensibly-chosen "best" address that could be stored and used by downstream processes.

Under the hood, this is a classic **entity-resolution pipeline**.

The stakes are high: if you cannot resolve addresses accurately, you cannot trace forced labor, spot sanctions evasion, or predict supply-chain bottlenecks. Downstream, everything is only as trustworthy as this resolution layer.

5. The data involved

You will be provided synthetic provider feeds/datasets of (company name, address components, timestamp). Schemas differ across providers — one spells out State / Country, another abbreviates admin_region / country. Volume starts small (tens of rows) and is designed to scale to millions; feeds arrive in **batch or stream**, and new providers can be added over time (N providers, $N \gg 2$).

Representative feeds (invented examples — shape only, no real data):

Provider A — schema: Company, Street Address, City, State, Country, Updated At

Company	Street Address	City	State	Country	Updated At
Arunima Art Shop	1234 Main St	Austin	Texas	United States of America	2022-01-01
Scents By Shannon	42 Answer Blvd	New York	New York	United States of America	2022-01-01
Emanuel's Empanada Emporium	234 Adelaide St	Toronto	Ontario	Canada	2022-01-01
Donut Dan	(blank)	(blank)	Connecticut	United States of America	2022-01-02
Nick's Knick Nacks	1776 Constitution Drive	Washington	District of Columbia	(blank)	2022-01-01
Jumping Joe's Juicy Juices	64A Wilbur Cres	Kansas City	Missouri	United States of America	2022-02-01
Jumping Joe's Juicy Juices	64 Wilbur Cres	Kansas City	Missouri	United States of America	2022-01-01

Provider B — *schema*: name, street_address, locality, admin_region, country, created_at

name	street_addresses	locality	admin_region	country	created_at
Masha's Matcha Munchies	1234 Main St	Austin	TX	US	2022-01-01
Scents By Shannon	12 Fun Blvd	Cambridge	MA	US	2022-01-01
Emanuel's Empanada Emporium	94 Ontario Blvd	Waterloo	ON	CA	2022-01-01
Donut dan	94 High Ridge Road	Stamford	CT	US	2022-01-01
Nicks Knick Nacks LLC	1776 Constitution Drive	Washington	DC	(blank)	2022-01-01

Target output — one resolved address per company, in a form downstream systems can consume:

```
{ "Arunima Art Shop": "1234 Main St, Austin, Texas, United States of America" }
```

6. Stakeholders

- **Producers:** the external data providers (simulated by advisor-generated synthetic feeds).
- **Consumers:** downstream analysts, risk / compliance teams, and the ML and graph systems that map supply chains and screen for sanctions and forced labor.
- **Accountable:** the data-platform / entity-resolution team that owns the "golden record" and must answer *"how do we know this is the right address?"*

7. What a "good" answer looks like

There is **no single ground-truth string** — a correct address has many valid representations — so "good" is judged on the quality of the *resolution*, not exact-match to a key.

A strong build tends to progress through four stages, and a team can go as deep as time allows:

1. **Read & unify** — ingest both feeds into a common schema.

2. **Deduplicate** — collapse obvious duplicates; naive exact-match first, then fuzzy.
3. **Aggregate, impute & clean** — apply cleaning and imputation strategies driven by what they notice in the data.
4. **Select the winner** — a final selection system that decides which address to keep (or synthesize) per company.

A team has nailed it if their output:

- **Deduplicates correctly** — each real-world entity appears once; true duplicates merge, genuinely distinct entities stay apart.
- **Chooses a defensible best address** per entity — reasoning beyond "most recent" (completeness, cross-source agreement).
- **Carries provenance** — for any resolved record, you can tell the *story* of that address: which sources contributed, what conflicted, and why this value won.
- **Scales and stays governed** as data grows and providers are added.
- **Is defensible under questioning** — the team can articulate over-merge vs. under-merge and precision / recall trade-offs, and what they would do with more time. (*Reasoning matters as much as code.*)

The answer key is **blind**: students resolve the feeds without seeing how they were generated.

8. Real-world messiness & trade-offs

- **No global address standard.** Western hierarchical addresses collapse across borders (Japan's block system; Latin-American landmark directions).
- **Recency ≠ quality.** The latest update can be the worst one.
- **Over-merge vs. under-merge.** Merge too eagerly and you fuse two real companies sharing a building; too timidly and one company splinters into many.
- **Legitimate ≠ real.** A corporate-registration suite is a real address but not a real place of business.
- **Scale.** Naive pairwise comparison is $O(N^2)$ and dies at millions of rows — forcing the idea of "blocking."
- **Untrusted, adversarial input.** Feeds are external — malformed rows, injection attempts ("Bobby Tables").
- **Async, heterogeneous, ever-growing providers.** Feeds arrive on different schedules, with different schemas, and new ones appear over time.

9. Component coverage

This use case naturally spans about **five of six** components, with **Data Engineering** and **Data Science / ML** as the twin cores.

#	Component	Fit	What students build
---	-----------	-----	---------------------

1	Apps + Lakebase	Open / intentional gap	Data generation is advisor-owned. If this component appears, it would be a student-built serving / search app over resolved entities — left open for teams to add.
2	Data Engineering	Core	The medallion ER pipeline: bronze (raw per-source) → silver (normalized common model) → gold (resolved golden records); incremental ingestion and scaling toward millions of rows.
3	Data Warehousing & AI/BI	Strong	Dashboards on match / dedup rates, data quality by source, country coverage, and "hive" detection (addresses linked to suspiciously many entities).
4	Genie & Agent Bricks	Good	Genie Q&A over the gold table ("how many entities share this address?", "tell the story of this record"); an agent to adjudicate ambiguous conflicts or the Translation Trap.
5	Data Science & Classic ML	Core	A supervised match / no-match classifier on candidate pairs (features: string similarity, token overlap, geospatial proximity, embeddings), and/or learned clustering for the golden-record step.
6	Governance & Monitoring	Thematic fit	Provenance is governance here: Unity Catalog lineage tells the address's story. Students build UC governance, AI Gateway usage, and compute / token cost

			monitoring.
--	--	--	-------------

"Incomplete on purpose" — level-up questions for students to take further. These are the natural extensions where teams get creative; each maps onto Databricks capabilities:

- **Scale:** How does this change with very large files (millions of rows)?
- **Iteration:** How would you design it for rapid iteration?
- **Production:** How would you deploy this into a real system — and how does prototype code differ from production code?
- **Testing:** How would you test it, given there is no single correct answer?
- **Provenance:** How would you augment the system to "tell the story" of a given address?
- **Security:** How would you handle untrusted input? (Bobby Tables.)
- **N suppliers:** How would you generalize from 2 feeds to N ($N \gg 2$)?
- **Hard conflicts:** How would you resolve genuinely ambiguous conflicts?
- **Asynchrony:** What if the deliveries arrive asynchronously?
- **AI:** How might you apply LLMs and embeddings — e.g. to the Translation Trap and the Formatting Illusion?

10. Data sensitivity

All data used in this exercise will be fully synthetic. Please do not attempt to use real addresses or actual geospatial data during this project.